Python Exercise: To-Do List App (CLI)

A progressive, hands-on exercise. Each level builds on the previous one, so work through them in order. Type the code yourself rather than copying — the mistakes are where the learning happens.

Prerequisites: Python 3.8+ installed. Comfortable with functions, loops, and `try` / `except` — do the Calculator exercise first if not.

Difficulty: Intermediate. This is the hardest of the three exercises.

Estimated time: 5–8 hours across all levels.

Learning objectives

By the end of this exercise you will be able to:

- Work with lists of dictionaries as an in-memory data store
- Design a command loop that maps user commands to actions
- Read and write JSON files, and handle a missing or corrupt file
- Use the `datetime` module for dates and overdue checks
- Build a proper command-line interface with `argparse`
- Separate data logic from presentation, and test the logic in isolation

---

Level 1 — Add and list

Goal

A program that lets you add tasks and see them listed.

Tasks

1. Create a file named `todo.py`.
2. Keep tasks in a list. Each task is a dictionary: `{"title": "...", "done": False}`.
3. Write `add_task(tasks, title)` — appends a new task.
4. Write `list_tasks(tasks)` — prints them numbered, starting at 1.
5. Build a command loop accepting `add`, `list`, and `quit`.

Starter code

```python
def add_task(tasks, title):
    """Append a new task. Returns the created task."""
    task = {"title": title, "done": False}
    tasks.append(task)
    return task


def list_tasks(tasks):
    if not tasks:
        print("  No tasks yet.")
        return
    for index, task in enumerate(tasks, start=1):
        mark = "x" if task["done"] else " "
        print(f"  {index}. [{mark}] {task['title']}")


def main():
    tasks = []
    while True:
        command = input("\n> ").strip().lower()

        if command == "add":
            title = input("Task: ").strip()
            # TODO: call add_task, confirm to the user
        elif command == "list":
            # TODO
            pass
        elif command in ("quit", "q"):
            break
        else:
            print("  Commands: add, list, quit")


if __name__ == "__main__":
    main()
```

Expected output

```
> add
Task: Buy milk
  Added: Buy milk

> add
Task: Call the bank
  Added: Call the bank

> list
  1. [ ] Buy milk
  2. [ ] Call the bank
```

Checkpoint questions

- Why `enumerate(tasks, start=1)` instead of `start=0`? What would the user see otherwise?
- Why store a dictionary rather than just the title string? What does that let you add later?

> **Hint (Level 1)** `enumerate` gives you the position and the item together. Without it you'd need a manual counter, which is one more thing to get wrong.

Level 2 — Complete and delete

Tasks

1. Write `complete_task(tasks, number)` — marks a task done.
2. Write `delete_task(tasks, number)` — removes it.

3. Handle out-of-range numbers. Asking to complete task 99 when there are 3 must not crash.
4. Handle non-numeric input at the number prompt.
5. Add a confirmation before deleting.

The off-by-one trap

The user sees task **1**, but Python stores it at index **0**. Every function taking a user-facing number must convert:

```
def complete_task(tasks, number):
    """Mark task `number` (1-based) as done. Returns True on success."""
    index = number - 1
    if not (0 <= index < len(tasks)):
        return False
    tasks[index]["done"] = True
    return True
```

Note the range check. `tasks[-1]` is valid Python — it means the *last* item — so if the user enters `0`, index becomes `-1` and you'd silently complete the wrong task. That's why `0 <= index` matters and is not redundant.

Test cases

Action	Expected behaviour
`done 2` with 3 tasks	Task 2 marked `[x]`
`done 99` with 3 tasks	"No task 99", no crash
`done 0` with 3 tasks	Rejected — not "completes the last task"
`done -1`	Rejected
`done abc`	"Please enter a number", no crash
`delete 1` then `list`	Remaining tasks renumbered from 1

Checkpoint questions

- After deleting task 1, what number does the old task 2 now have? Is that a problem for the user?
- Should `complete_task` print an error, or return `False` and let the caller print? Justify your answer.

Level 3 — Make it persist

Everything vanishes when the program exits. Fix that with JSON.

Tasks

1. Write `save_tasks(tasks, path)` — writes the list to a JSON file.
2. Write `load_tasks(path)` — reads it back. Returns an empty list if the file doesn't exist.
3. Handle a **corrupt** file. If the JSON is malformed, don't crash — warn the user and start fresh.
4. Save after every change, not just on quit. Ask yourself why that matters.

Starter code

```python
import json
import os

DATA_FILE = "tasks.json"


def save_tasks(tasks, path=DATA_FILE):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(tasks, f, indent=2, ensure_ascii=False)


def load_tasks(path=DATA_FILE):
    if not os.path.exists(path):
        return []
    try:
        with open(path, encoding="utf-8") as f:
            data = json.load(f)
    except json.JSONDecodeError:
        print(f"  Warning: {path} is corrupt. Starting with an empty list.")
        return []
    if not isinstance(data, list):
        print(f"  Warning: {path} has an unexpected shape. Starting fresh.")
        return []
    return data
```

Try to break it

- Delete `tasks.json` and run the program. Does it start cleanly?
- Put the text `not json at all` in `tasks.json`. Does it warn instead of crashing?
- Put `{"a": 1}` in it — valid JSON, wrong shape. Is that handled?
- Add a task with Arabic or emoji in the title, save, reload. Does it survive? (This is what `ensure_ascii=False` is for.)

Checkpoint questions

- What does `ensure_ascii=False` change in the saved file? Open it in a text editor with and without.
- If the program is killed halfway through `json.dump`, what state is the file in? How would you make saving safer? (Search for "atomic write".)

> **Hint (Level 3)** The safe pattern is to write to a temporary file, then rename it over the original. Renaming is atomic on most filesystems, so the real file is either the old version or the new one — never a half-written mix.

Level 4 — Priorities and due dates

Tasks

1. Extend each task with `priority` (`high`, `medium`, `low`) and `due` (an ISO date string like `2026-09-15`, or `None`).
2. Validate the date on entry. Reject `2026-13-45`.
3. Mark overdue tasks in the listing.
4. Add sorting: by priority, by due date, or by creation order.
5. Add filtering: show only pending, only completed, or only overdue.

Working with dates

```python
from datetime import date, datetime


def parse_due(text):
    """Parse YYYY-MM-DD. Returns a date, or None if the text is empty.
    Raises ValueError if the text is present but not a valid date."""
    text = text.strip()
    if not text:
        return None
    return datetime.strptime(text, "%Y-%m-%d").date()


def is_overdue(task):
    if task["done"] or not task.get("due"):
        return False
    return date.fromisoformat(task["due"]) < date.today()
```

Backwards compatibility

Your existing `tasks.json` has entries with no `priority` or `due` key. Loading it will raise `KeyError`. Two ways to deal with it:

- Use `task.get("priority", "medium")` everywhere instead of `task["priority"]`
- Write a migration that adds the missing keys on load

Implement the second. Handling old data formats is a real and permanent part of software work, and doing it once here is worth more than reading about it.

Checkpoint questions

- Why store dates as ISO strings in JSON rather than `date` objects? What does `json.dump` do if you hand it a `date`?
- ISO format sorts correctly as plain text — `"2026-01-05" < "2026-02-01"` is `True`. Would `"05/01/2026"` sort correctly?

Level 5 — A real command-line interface

So far the program is interactive. Now make it scriptable:

```
python3 todo.py add "Buy milk" --priority high --due 2026-09-15
python3 todo.py list --filter pending --sort due
python3 todo.py done 2
python3 todo.py delete 3 --yes
```

Tasks

1. Rewrite the entry point using `argparse` with subcommands.
2. Keep the interactive mode available via `python3 todo.py` with no arguments.
3. Return proper exit codes: `0` on success, `1` on failure. Scripts and CI depend on these.
4. Add `--file` so the data file path can be overridden.

Starter code

```python
import argparse
import sys


def build_parser():
    parser = argparse.ArgumentParser(description="A simple to-do list.")
    parser.add_argument("--file", default=DATA_FILE, help="path to the data file")
    subs = parser.add_subparsers(dest="command")

    p_add = subs.add_parser("add", help="add a task")
    p_add.add_argument("title")
    p_add.add_argument("--priority", choices=["high", "medium", "low"],
                       default="medium")
    p_add.add_argument("--due", help="due date as YYYY-MM-DD")

    p_list = subs.add_parser("list", help="list tasks")
    p_list.add_argument("--filter", choices=["all", "pending", "done", "overdue"],
                        default="all")
    p_list.add_argument("--sort", choices=["created", "priority", "due"],
                        default="created")

    p_done = subs.add_parser("done", help="mark a task complete")
    p_done.add_argument("number", type=int)

    # TODO: a 'delete' subcommand with a --yes flag to skip confirmation

    return parser


def main(argv=None):
    args = build_parser().parse_args(argv)
    tasks = load_tasks(args.file)

    if args.command is None:
        return interactive_mode(tasks, args.file)

    # TODO: dispatch on args.command, return 0 or 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Checkpoint questions

- Run `python3 todo.py --help` and `python3 todo.py add --help`. What did argparse generate for free?
- Why does `main()` take `argv=None` rather than reading `sys.argv` directly? (Hint: it makes Level 6 far easier.)
- What happens with `--due bananas`? Where should that be caught?

Level 6 — Tests

Tasks

1. Create `test_todo.py`.
2. Test the data functions — `add_task`, `complete_task`, `delete_task` — with no file access at all.
3. Test save/load using a temporary file, so tests never touch your real `tasks.json`.
4. Test the corrupt-file path.
5. Test `main()` end-to-end by passing an argument list directly.

Starter code

```python
import json
import os
import tempfile
import unittest

from todo import add_task, complete_task, delete_task, save_tasks, load_tasks


class TestTaskOperations(unittest.TestCase):

    def setUp(self):
        self.tasks = []
        add_task(self.tasks, "First")
        add_task(self.tasks, "Second")

    def test_add_appends(self):
        self.assertEqual(len(self.tasks), 2)
        self.assertEqual(self.tasks[0]["title"], "First")
        self.assertFalse(self.tasks[0]["done"])

    def test_complete_valid(self):
        self.assertTrue(complete_task(self.tasks, 1))
        self.assertTrue(self.tasks[0]["done"])

    def test_complete_out_of_range(self):
        self.assertFalse(complete_task(self.tasks, 99))

    def test_complete_zero_is_rejected(self):
        # 0 would become index -1 and silently hit the LAST task
        self.assertFalse(complete_task(self.tasks, 0))
        self.assertFalse(self.tasks[-1]["done"])

    def test_delete_renumbers(self):
        delete_task(self.tasks, 1)
        self.assertEqual(len(self.tasks), 1)
        self.assertEqual(self.tasks[0]["title"], "Second")


class TestPersistence(unittest.TestCase):

    def setUp(self):
        handle, self.path = tempfile.mkstemp(suffix=".json")
        os.close(handle)
        os.unlink(self.path)          # we want the path, not the file

    def tearDown(self):
        if os.path.exists(self.path):
            os.unlink(self.path)

    def test_missing_file_returns_empty(self):
        self.assertEqual(load_tasks(self.path), [])

    def test_round_trip(self):
        tasks = [{"title": "Test", "done": False}]
        save_tasks(tasks, self.path)
        self.assertEqual(load_tasks(self.path), tasks)

    def test_corrupt_file_does_not_raise(self):
        with open(self.path, "w") as f:
            f.write("this is not json")
        self.assertEqual(load_tasks(self.path), [])

    def test_unicode_survives(self):
        tasks = [{"title": "شراء الحليب", "done": False}]
        save_tasks(tasks, self.path)
        self.assertEqual(load_tasks(self.path)[0]["title"], "شراء الحليب")


if __name__ == "__main__":
    unittest.main()
```

Run with `python3 -m unittest test_todo.py -v`.

Checkpoint questions

- Why does `setUp` run before every test rather than once for the whole class? What would break if tests shared state?
- `test_complete_zero_is_rejected` exists because of a specific Python behaviour. Which one, and what bug would it have caused?

Common errors and what they mean

Error	Usual cause
`IndexError: list index out of range`	Missing bounds check before indexing
Wrong task completed when entering `0`	Negative index wrapped to the end — check `0 <= index`
`KeyError: 'priority'`	Loading old data that predates the field — migrate or use `.get()`
`json.decoder.JSONDecodeError`	The data file is corrupt or empty — catch it
`TypeError: Object of type date is not JSON serializable`	Passed a `date` to `json.dump` — store `.isoformat()`
Arabic saved as `\u0634\u0631\u0627\u0621`	Missing `ensure_ascii=False`
Changes lost on exit	Saved only on quit, and the program crashed first
`FileNotFoundError` on first run	`load_tasks` doesn't check whether the file exists

Self-assessment

You have finished when you can answer yes to all of these:

- ☐ The app never crashes — bad numbers, bad dates, missing or corrupt file
- ☐ Data survives restarts, including non-English text
- ☐ Data functions contain no `print()` and no `input()`
- ☐ Both `python3 todo.py add "x"` and interactive mode work
- ☐ Tests pass and use a temp file, never your real `tasks.json`
- ☐ You can explain the off-by-one issue and where you guard against it

---

Reference solution — Levels 1 to 3

Do not read this until you have made a serious attempt. Struggling with a bug for twenty minutes teaches more than reading a working answer.

```
"""A simple command-line to-do list."""

import json
import os

DATA_FILE = "tasks.json"


# ---------- data operations (no printing, no input) ----------

def add_task(tasks, title):
    """Append a new task. Returns the created task."""
    task = {"title": title, "done": False}
    tasks.append(task)
    return task
```

```python
def complete_task(tasks, number):
    """Mark task `number` (1-based) done. Returns True on success."""
    index = number - 1
    if not (0 <= index < len(tasks)):
        return False
    tasks[index]["done"] = True
    return True


def delete_task(tasks, number):
    """Remove task `number` (1-based). Returns the removed task, or None."""
    index = number - 1
    if not (0 <= index < len(tasks)):
        return None
    return tasks.pop(index)


# ---------- persistence ----------

def save_tasks(tasks, path=DATA_FILE):
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(tasks, f, indent=2, ensure_ascii=False)
    os.replace(tmp, path)          # atomic: never a half-written file


def load_tasks(path=DATA_FILE):
    if not os.path.exists(path):
        return []
    try:
        with open(path, encoding="utf-8") as f:
            data = json.load(f)
    except json.JSONDecodeError:
        print(f"  Warning: {path} is corrupt. Starting with an empty list.")
        return []
    if not isinstance(data, list):
        print(f"  Warning: {path} has an unexpected shape. Starting fresh.")
        return []
    return data


# ---------- presentation ----------

def render(tasks):
    if not tasks:
        print("  No tasks yet.")
        return
    for index, task in enumerate(tasks, start=1):
        mark = "x" if task["done"] else " "
        print(f"  {index}. [{mark}] {task['title']}")


def read_number(prompt):
    """Return a whole number, or None if the input was not one."""
    text = input(prompt).strip()
    try:
        return int(text)
    except ValueError:
        print("  Please enter a number.")
        return None


# ---------- command loop ----------

HELP = "Commands: add, list, done, delete, help, quit"


def main():
    tasks = load_tasks()
    print("=== To-Do List ===")
    print(HELP)

    while True:
```

```
        command = input("\n> ").strip().lower()

        if command == "add":
            title = input("Task: ").strip()
            if not title:
                print("  Title cannot be empty.")
                continue
            add_task(tasks, title)
            save_tasks(tasks)
            print(f"  Added: {title}")

        elif command == "list":
            render(tasks)

        elif command == "done":
            render(tasks)
            number = read_number("Complete which number? ")
            if number is None:
                continue
            if complete_task(tasks, number):
                save_tasks(tasks)
                print("  Marked complete.")
            else:
                print(f"  There is no task {number}.")

        elif command == "delete":
            render(tasks)
            number = read_number("Delete which number? ")
            if number is None:
                continue
            index = number - 1
            if not (0 <= index < len(tasks)):
                print(f"  There is no task {number}.")
                continue
            confirm = input(f"  Delete '{tasks[index]['title']}'? (y/n): ")
            if confirm.strip().lower() in ("y", "yes"):
                removed = delete_task(tasks, number)
                save_tasks(tasks)
                print(f"  Deleted: {removed['title']}")
            else:
                print("  Cancelled.")

        elif command in ("help", "?"):
            print(HELP)

        elif command in ("quit", "q", "exit"):
            break

        else:
            print(f"  Unknown command. {HELP}")

    print("Saved. Goodbye.")


if __name__ == "__main__":
    main()
```

Points worth noticing in this solution:

- The file is split into three sections — data, persistence, presentation. The data functions never print, which is exactly why they're trivial to test.
- `save_tasks` writes to a temp file then calls `os.replace`. A crash mid-write leaves the old file intact rather than a corrupt one.
- Every function taking a user number does the `0 <= index < len(tasks)` check. Missing the `0 <=` half is the classic bug here.
- `add_task` returns the task it created, and `delete_task` returns the one it removed. Returning something useful costs nothing and often saves a lookup later.
- Saving happens after every change, so a crash loses at most nothing.